

COMP232 Cyber Security — Assignment 2

COMP232 | April 27, 2026

Zhibin Jiang

1 Comparison of Message Authentication Methods

The three methods below are compared across **integrity**, **authentication**, and **non-repudiation**, as studied in Labs 5 and 6.

1.1 Hash Functions (SHA-256)

Purpose & assumptions. Maps an arbitrary message to a fixed 256-bit digest. Used solely for *integrity checking*: any bit-flip in the input changes the digest entirely. Assumes collision resistance and pre-image resistance; no key is required.

Security properties.

- **Integrity:** Receiver recomputes $\text{SHA256}(m)$ and compares with the received digest.
- **Authentication:** *Not provided* — anyone can compute the hash; no proof of origin.
- **Non-repudiation:** *Not provided* — no secret links the digest to a specific sender.

Pros: Fast, keyless, compact output (32 bytes).

Cons: Integrity only; must be combined with a keyed scheme for authentication.

1.2 Digital Signatures (RSA/DSA)

Purpose & assumptions. As practised in Lab 5, the sender computes $h=H(m)$ and signs it with their *private key*: $\sigma = \text{RSA}_{\text{priv}}(h)$. The receiver verifies using the sender's *public key*. Assumes hardness of integer factorisation (RSA) and collision resistance of the hash; requires a trusted public key.

Security properties.

- **Integrity:** Signature verification fails if m is altered.
- **Authentication:** Only the private-key holder can produce a valid σ .
- **Non-repudiation:** Any third party can verify σ with the public key; the sender cannot deny signing.

Pros: Provides all three properties; public verification needs no shared secret.

Cons: Computationally expensive; depends on PKI for key distribution.

1.3 HMAC-SHA256

Purpose & assumptions. As studied in Lab 6, HMAC mixes a shared secret key K with the message through two nested hash calls, producing a 256-bit tag. Assumes SHA-256 pseudo-randomness and that both parties have already established K securely (e.g., via Diffie-Hellman from Lab 4).

Security properties.

- **Integrity:** Receiver recomputes $\text{HMAC}(K, m)$ and checks it matches the tag.
- **Authentication:** Only parties holding K can generate or verify the tag.
- **Non-repudiation:** *Not provided* — since K is shared, either party could have produced the tag; no third-party proof is possible.

Pros: Fast; stronger than a plain keyed hash (resistant to length-extension attacks); widely deployed (TLS, JWTs).

Cons: Requires prior secure key distribution; no non-repudiation.

1.4 Summary

Table 1: Security properties of each method.

Method	Integrity	Auth.	Non-rep.
SHA-256	✓	×	×
Digital Signature	✓	✓	✓
HMAC-SHA256	✓	✓	×

2 MD5 Collision Attack Experiment

The experiment was conducted on a local Ubuntu machine using the `md5collgen` tool (v1.5, by Marc Stevens) from the SEED Labs 2.0 setup. The chosen prefix was the ASCII string "hello" (6 bytes, including the new-line appended by `echo`).

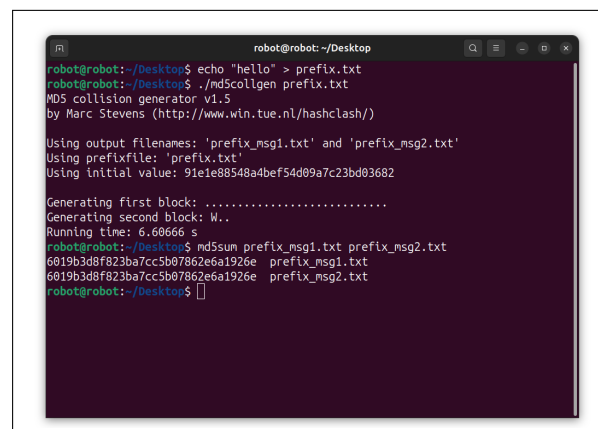
2.1 Task 1 — Question 1

If the prefix file length is not a multiple of 64 bytes, what happens?

MD5 processes input in 64-byte (512-bit) blocks. When the prefix is shorter than 64 bytes, `md5collgen` pads it to a full block using the standard MD5 padding scheme: a `0x80` byte is appended, then `0x00` bytes up to byte 55, and finally the 64-bit message length in bytes 56–63. The MD5 compression function is then applied to this padded block to produce an *Intermediate Hash Value* (IHV), which serves as the starting point for the collision computation. The tool operates normally for any prefix up to 64 bytes.

In our run, the 6-byte prefix "hello\n" was padded to 64 bytes, yielding the IV `91e1e88548a4bef54d09a7c23bd03682` (Figure 1). The tool then appended its 128-byte collision block to produce two output files of 192 bytes each.

Figure 1 shows the tool running successfully in the VM.



```
robot@robot: ~/Desktop
robot@robot: ~/Desktop$ echo "hello" > prefix.txt
robot@robot: ~/Desktop$ ./md5collgen prefix.txt
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'prefix_msg1.txt' and 'prefix_msg2.txt'
Using prefix file: 'prefix.txt'
Using initial value: 91e1e88548a4bef54d09a7c23bd03682

Generating first block: .....
Generating second block: W..
Running time: 6.68666 s
robot@robot: ~/Desktop$ md5sum prefix_msg1.txt prefix_msg2.txt
6019b3d8f023ba7cc5b07062e6a1926e prefix_msg1.txt
6019b3d8f023ba7cc5b07062e6a1926e prefix_msg2.txt
robot@robot: ~/Desktop$
```

Figure 1: Running `md5collgen` (SEED Labs 2.0) with prefix "hello" on local Ubuntu.

2.2 Task 1 — Question 3

Are the 128 bytes generated by `md5collgen` completely different for the two output files?

No. The 128-byte collision block generated by `md5collgen` is *partially* different — only a handful of bytes differ, while the vast majority are identical. Figure 2 shows the byte-level diff from the VM.

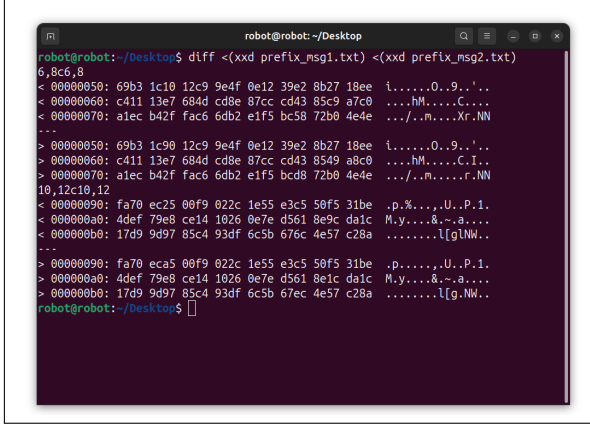


Figure 2: Hexdump diff between `prefix_msg1.txt` and `prefix_msg2.txt` — differing bytes highlighted.

The two output files are each 192 bytes: a 64-byte padded prefix block followed by a 128-byte collision block. A byte-level comparison (Figure 2) reveals that exactly **7 bytes** differ within the collision block (spanning bytes 64–191), while the remaining 121 bytes are identical:

Table 2: Differing bytes between the two output files.

Offset	msg1	msg2	Diff
83	0x10	0x90	+0x80
109	0xc9	0x49	-0x80
110	0xa7	0xa8	+0x01 [†]
123	0x58	0xd8	+0x80
147	0x25	0xa5	+0x80
173	0x9c	0x1c	-0x80
187	0x6c	0xec	+0x80

[†] Carry: $0xc9 + 0x80 = 0x149$, overflows byte 110 by 1.

Six of the seven changed bytes differ by exactly `0x80` (a single high-bit flip). The seventh byte (offset 110) changes by `+0x01` as an arithmetic carry: the 32-bit word at offsets 108–111 changes by `+0x8000`, which overflows byte 109 and propagates a carry into byte 110. The remaining 121 bytes of the collision block are byte-for-byte identical. Despite only these minor perturbations, both files produce the same MD5 hash `6019b3d8f823ba7cc5b07862e6a1926e`. This illustrates the Wang et al. differential attack underlying `md5collgen`: the tool engineers two 128-byte blocks whose difference cancels exactly through all four MD5 rounds, leaving both files with an identical final hash state.

3 Four-Party Diffie-Hellman Key Exchange

This section extends the 2-party DH protocol (Lab 4’s `DH.py`) and the 3-party protocol (`DH3.py`) to four participants: **Alice**, **Bob**, **Carol**, and **Dave**. The goal is for all four to derive the same shared secret $K = g^{abcd} \bmod p$.

3.1 Design: Extending 2-Party DH to Four Parties

In the 2-party DH, Alice (a , $A = g^a$) and Bob (b , $B = g^b$) each raise the other’s public key to their own private exponent: $K = B^a = A^b = g^{ab}$. The 3-party variant in `DH3.py` extends this with one extra round: a partial key g^{ab} is relayed and raised to c , giving g^{abc} for all three.

For four parties, a naïve cyclic relay fails—each exponent would be applied more than once, yielding $g^{a^2b^2c^2d^2}$ instead of g^{abcd} . The correct approach is a **binary-tree** structure:

- Round 1 (pairwise):** Pair (A,B) compute $K_{AB} = g^{ab}$; pair (C,D) compute $K_{CD} = g^{cd}$.
- Round 2 (cross-exchange):** Each pair raises its partial key to both exponents of the other pair, obtaining g^{abcd} .

Each private exponent appears exactly once in the final product, which is why the tree structure works where the cyclic relay does not.

3.2 Protocol Description

Round 1 — Pairwise DH. Using the same (g, p) parameters, each pair runs an independent 2-party DH:

$$K_{AB} = B^a \bmod p = g^{ab}, \quad K_{CD} = D^c \bmod p = g^{cd}$$

Both partners in each pair independently verify they reach the same value (Alice checks B^a , Bob checks A^b , etc.).

Round 2 — Cross-exchange. The two partial keys are raised to the remaining pair’s private exponents:

$$\text{Alice \& Bob: } (K_{AB}^c)^d = (g^{ab})^{cd} = g^{abcd}$$

$$\text{Carol \& Dave: } (K_{CD}^a)^b = (g^{cd})^{ab} = g^{abcd}$$

All four parties obtain the same value $K = g^{abcd}$. Since exponentiation is commutative in $\mathbb{Z}_p^*$, the order in which exponents are applied does not matter.

3.3 Implementation

The implementation (`DH4.py`) uses Python’s `cryptography` library to generate a 1024-bit DH group (g, p) and the four key pairs. The core computation is:

```
# Round 1: pairwise DH
K_AB = pow(B, a, p) # = g^(ab); Bob verifies:
           pow(A, b, p)
K_CD = pow(D, c, p) # = g^(cd); Dave verifies:
           pow(C, d, p)

# Round 2: cross-exchange
K_alice = pow(pow(K_AB, c, p), d, p) # (g^ab)^c
           ^d = g^(abcd)
K_bob   = pow(pow(K_AB, c, p), d, p)
K_carol = pow(pow(K_CD, a, p), b, p) # (g^cd)^a
           ^b = g^(abcd)
K_dave  = pow(pow(K_CD, a, p), b, p)

assert K_alice == K_bob == K_carol == K_dave #
           all match
```

The shared integer K is converted to bytes and passed through `HKDF-SHA256` (32 bytes,

`info=b'4party-DH-handshake')` to produce an AES-256 key. Bob then encrypts a test message in AES-256-CBC mode; Alice decrypts it to confirm the shared key.

3.4 Verification

Running `python3 DH4.py` with freshly generated keys produces:

```

=== Round 1: Pairwise DH ===
Alice computes K_AB = B^a mod p = g^(ab)
Bob verifies K_AB = A^b mod p = g^(ab) [OK]
Carol computes K_CD = D^c mod p = g^(cd)
Dave verifies K_CD = C^d mod p = g^(cd) [OK]

=== Round 2: Cross-Exchange ===
Alice: pow(pow(K_AB, c, p), d, p) = g^(abcd)
Bob: pow(pow(K_AB, c, p), d, p) = g^(abcd)
Carol: pow(pow(K_CD, a, p), b, p) = g^(abcd)
Dave: pow(pow(K_CD, a, p), b, p) = g^(abcd)

=== Verification ===
All four shared secrets match: True

Shared secret K = g^(abcd) (hex, 128 bytes):
376383894367af133fd477e900cdc4ba
79bdcfb0c56681dfbb368b9182265562
274a4d6c404b723f502a42aba98a622f
fd3275d871ab9c170e480f7697a135ce
6bd6fb098e188e6accec9bd18cca56f4
05c7438cced0ac758a2b3931b077a00c
1c4953e4d2a2dab8cf3fa29acb9bf346
ef455ad41856bb223d5e2e94105cc6a3

Derived AES-256 key (HKDF-SHA256, 32 bytes):
dcb542ab8ae86d0faadd43b948eb7c0f
29b9ed853b4b496eb608ab195925a34d

Plaintext (Bob): b'Privacy Security|Bob -> Alice
via 4-party DH!'
Ciphertext (AES-256-CBC, 48 bytes):
06ecf6d69506fafb6921f256a8f87b90
efd7f9f44df8b2a5596f65c0fab8a60a
eca5ce9e06d44309dea0109de816df98
IV: bacdd347b571487901152f790466d49d

Decrypted (Alice): b'Privacy Security|Bob ->
Alice via 4-party DH!'

```

The `assert` on line 56 of `DH4.py` passes without exception, confirming all four computed values are identical. Alice successfully decrypts Bob's ciphertext, providing end-to-end evidence that the shared secret $K = g^{abcd}$ was correctly established by all four parties.

3.5 Security Considerations

The binary-tree protocol inherits the security guarantees of 2-party DH. An eavesdropper who intercepts all public keys (A, B, C, D) and both intermediate values ($K_{AB} = g^{ab}$, $K_{CD} = g^{cd}$) still cannot compute g^{abcd} without solving the **Computational Diffie-Hellman (CDH)** problem—believed to be as hard as the discrete logarithm in $\mathbb{Z}_p^*$. With a 1024-bit prime p , this is computationally infeasible for a passive adversary.

One limitation is that the protocol as simulated assumes a *trusted broadcast channel*: in practice, each party would need to authenticate the intermediate values it receives (e.g., via digital signatures) to prevent man-in-the-middle attacks, in the same way that 2-party DH requires authenticated channels.